• 2025 •

CODING STANDARDS

GAMIFIED FINANCIAL VISUALIZER

For

Demo 3

Presented by :

CodeBlooded

Introduction

This Coding Standards & Engineering Playbook defines the conventions we follow across the Gamified Finance Capstone Project to keep the codebase consistent, readable, testable, and secure. A shared standard reduces technical debt, makes reviews objective, speeds up onboarding, and lets the team ship changes with confidence.

Scope. These standards apply to every part of the system: the web frontend (React + Tailwind), the primary backend APIs (Node/Express, TypeScript), the AI microservice (FastAPI, Python), middleware, scripts, database migrations (PostgreSQL), caching (Redis), and all tests and CI/CD workflows.

Audience. All contributors—students, reviewers, and future maintainers. Reviewers should use this document as the baseline for PR feedback; contributors should treat it as the single source of truth.

Guiding principles.

- Clarity over cleverness: small, focused modules with clear names.
- Typed and validated boundaries: strong types and input validation at API edges.
- Security by default: least privilege, no secrets in code, safe error handling.
- Performance and reliability: paginated I/O, efficient queries, measured with logs/metrics.
- Accessibility and internationalization: inclusive UI, key-based strings.
- Consistency enforced by tools: linters, formatters, tests, and CI gates.

What this covers.

- Project/file structure and naming
- Code formatting, linting, and comments
- Error handling and logging
- Version control and API versioning
- Testing strategy and coverage expectations
- Dependency and environment management
- Performance, security, and accessibility guidelines
- Documentation practices and change control

Compliance & evolution. CI enforces formatting, linting, and tests on every PR. Any intentional deviation must be justified in the PR description and, when architectural, captured as an ADR. This document evolves with the system; propose updates via PR so the standards stay useful and current.

By following these guidelines, we keep the platform scalable, secure, and easy to extend as new features land and existing ones mature.

Project Structure

```
gamified-financial-visualizer/
├── frontend/                    # Frontend (React)
│   ├── public/
│   │   ├── models/              # 3d models for AR
│   ├── src/
│   │   ├── assets/              # Images, fonts, etc.
│   │   ├── components/          # Reusable UI components
│   │   ├── pages/               # Page-level components
│   │   ├── layouts/             # React page layouts
│   │   └── App.jsx              # Main app entry
│   └── ...
├── api-backend/                 # Backend (Node.js)
│   ├── config/                  # DB/config files
│   ├── events/
│   ├── modules/                 # Auth, Transactions, Community, etc.
│   │   ├── service1/
│   │   ├── service2/
│   ├── jobs/
│   ├── queues/
│   ├── schedulars/
│   ├── tests/                   # Unit and integration tests
│   └── server.ts                # Entry point
├── documentation/               # Project documentation
└── .github/                     # GitHub workflows/CI
```

File Naming Conventions

General Rules:

- Use kebab-case for file and folder names (e.g., dashboard-component.jsx, api-utils.js).
- Avoid spaces, special characters, or uppercase letters in filenames in the api-backend.
- Match component names with file names (e.g., UserProfile.jsx → user-profile.jsx).

Frontend (React):

- Components: [component-name].jsx (PascalCase for component names, kebab-case for files).
- Tests: [component-name].test.js (Jest/React Testing Library).

Backend (Node.js/REST API):

- Models: [model-name].ts (lowercase for model names).
- Routes: [resource]Routes.ts (e.g., authRoutes.ts).
- Services: [service].service.ts (e.g, auth.service.ts).

Error Handling

Backend

Central error middleware returns a consistent envelope:

{ "error": { "code": "RESOURCE_NOT_FOUND", "message": "Goal not found", "details": {...}, "traceId": "<reqId>" } }

- Map common cases: **VALIDATION_ERROR(400), UNAUTHENTICATED(401), FORBIDDEN(403), NOT_FOUND(404), CONFLICT(409), RATE_LIMITED(429), INTERNAL(500)**.
- Never leak stack traces in production; always include a trace ID.
- Wrap async handlers; use zod for request validation at boundaries.

Frontend

- Show friendly toasts/banners; never block UI with alert() for expected errors.
- Handle network errors (timeout, offline) and show retry options.

Logging

- Use winston (server) with JSON logs; console (client) only in dev.
- Include: **level, timestamp, traceId, userId?, route, durationMs**.
- Mask PII; never log secrets, tokens, or passwords.
- Levels: **trace** (dev only), **debug, info, warn, error, fatal**.
- Correlate logs with a request ID (set in middleware).

Verion Control

Branch Naming:

- subsystem/feature/[description] (e.g., frontend/feature/user-auth).
- subsystem/bugfix/[description] (e.g., backend/bugfix/login-error).

Commits:

- Conventional Commits (e.g., feat: add login button, fix: resolve API 500 error).

Pull Requests:

- Require 1 approval before merging to main.
- Link PRs to GitHub Issues.

Testing

- **Jest** everywhere. Targets & patterns:
  - **Frontend**: React Testing Library + @testing-library/user-event.
  - **Backend**: unit test services with mocked pool.query; route tests can hit controllers with supertest optionally.
- Minimum coverage (backend): lines 80%, branches 70% (raises over time).
- Test names: “should … when …”.
- Seeded test data via factory helpers; avoid real network calls.
- Snapshot tests are allowed for stable, visual components only.

Linting & Formatting

ESLint:

- Airbnb JavaScript/React style guide (extends eslint-config-airbnb).
- Custom rules:
  - Max line length: 100 characters.
  - Always use semicolons.
  - Prefer const over let where possible.
- Run lint checks before commits (Husky pre-commit hook).

Prettier:

- Single quotes, trailing commas, and 2-space indentation.
- .prettierrc config:

```
{
 "singleQuote": true,
 "trailingComma": "es5",
 "tabWidth": 2
}
```

Dependency Management

- Use npm workspaces with a single root lockfile.
- Pin with caret (^) and rely on Dependabot/Renovate for updates.
- Before adding a package: check size, maintenance, license, and overlap with existing libs.
- Frontend networking: native Fetch only (no Axios).
- Regular npm audit in CI (non-blocking for low severity).

API Desing Guidelines

- REST first, versioned: **/api/v1/....**
- Nouns & plurals: **/accounts/{id}/transactions**.
- IDs are opaque. Use UUIDs on public endpoints.
- Idempotency: POSTs that transfer money or create one-time actions accept the **Idempotency-Key** header.
- Request/response contracts validated by zod. Document with OpenAPI.
- Response envelope:

```
{ "data": {...}, "meta": { "traceId": "<id>", "paging": {...} } }
```

- Use 201 + **Location** on resource creation; 204 on successful deletes.

Documentation

- Each feature folder includes a **README.md** (purpose, API, notable decisions).
- JSDoc/TSDoc for exported functions/types.
- Architecture Decision Records (ADRs) for non-trivial choices (**/docs/adr/**).
- User manual and screenshots live in **/documentation** and are updated per release.

Performance Guidelines

- Database:
  - parameterized queries only; create indexes for frequent filters; avoid N+1 by batching.
- API:
  - paginate lists; stream big downloads; cache hot reads in Redis with sensible TTLs and cache keys (**entity:v1:account:<id>**).
- Frontend:
  - memoize heavy components; avoid re-renders (use **useMemo, useCallback** judiciously).
- Images/models: lazy-load; compress; use CDN paths in production.
- 3D/AR:
  - Limit draw calls; LODs for complex scenes; suspend expensive effects off-screen.

Concurrancy & Parallelism

- Node is single-threaded—avoid CPU spikes on the request path.
- Use DB transactions for multi-step writes; choose **READ COMMITTED** by default, escalate if needed.
- For long tasks (e.g., AI insights, media processing), enqueue with BullMQ (Redis) and notify via webhook/socket.
- Guard against race conditions: optimistic locking (updated_at), idempotency keys for retried requests.

Security Guidelines

- OWASP Top 10 aware coding; threat model new endpoints.
- Auth: JWT (short-lived) + refresh flow; store access token in memory, refresh in httpOnly cookie where applicable.
- Input validation with **zod**; output encoding for HTML contexts.
- CORS: explicit origins per environment.
- Rate limiting & Helmet middleware enabled.
- Secrets in environment variables only; never in repo. Use platform secret manager in prod.
- Passwords: bcrypt with salt + server-side pepper (per assignment work). Never log.

CI/CD

- Unified **GitHub Actions** pipeline:
  - Install (cache),
  - Lint,
  - Test (frontend & backend),
  - Build,
  - Upload artifacts,
  - (Optional) Deploy.
- Start services for tests as needed (Postgres via **services:** ; Redis if required).
- Block merges on failing CI. Coverage threshold enforced.
- On deploy: run DB migrations first; health check endpoint must pass.

Code Review

- Reviewer checklist:
  - Correctness, tests, error handling, logging, security, accessibility, performance, i18n keys, dark mode states.
  - No TODOs that block; no commented-out code.
  - Names & comments clear; functions cohesive.
- Approve only if you'll own the code later.

Tooling

- ESLint + Prettier. Base configs:
  - Frontend: **eslint-plugin-react, react-hooks, jsx-a11y**.
  - Backend: **@typescript-eslint**.
- Husky pre-commit: **lint-staged** for **eslint --fix** + **prettier --write** + j**est -o --bail**.
- EditorConfig at repo root.
- Commit hooks never run network or DB calls.

Database Practices

- Naming: snake_case, plural table names (**user_points, goal_progress)**.
- Primary keys: **id UUID DEFAULT gen_random_uuid().**
- Timestamps: **created_at, updated_at** (**DEFAULT now()** ; trigger to update).
- Foreign keys with **ON DELETE RESTRICT** (or **CASCADE** when safe).
- Migrations: keep ordered SQL files under **api-backend/src/db/migrations/**. One migration per change, reversible where possible.
- Use views/materialized views for heavy aggregations; keep business rules in services, not the DB, except vetted triggers (e.g., XP accrual).

Environment-Specific Guidelines

- **NODE_ENV** in {development, test, staging, production}.
- Config loader selects **.env** file and merges with defaults; fails fast when required vars are missing.
- Dev: verbose logs, pretty errors; Prod: info+ logs, strict CORS, secure cookies.
- Separate DBs per env; never point local to prod.
- Feature flags for risky launches.

UI/UX Guidelines

- Tailwind is the default styling. Keep spacing scale consistent (4/8px rhythm).
- **Color system:** blues + neutrals; dark mode background **#0E171F**. Respect prefers-color-scheme, but the final source of truth is the Settings toggle.
- **Accessibility:**
  - Proper aria- attributes, visible focus, 4.5:1 contrast minimum.
  - Keyboard navigation for dropdowns/menus/modals.
- Components are resilient: loading, empty, and error states.
- Animations via Framer Motion—subtle and interruptible.
- Icons via **react-icons** (consistent sizes).
- **3D/AR:**
  - match Blender lighting (dual suns), keep exposure consistent across viewers.

Localization & Internationalization

- Key-based strings (**learn.module.title**) with ICU formatting.
- Keep copy in **/frontend/src/i18n/**.
- No hardcoded text in components; pass labels as props for reusable primitives.
- Date/number formatting via **Intl.***.

Deprecated Code

- Mark with **/** @deprecated reason. Removal: YYYY-MM-DD */** and a deprecation issue.
- Hide from UI behind flag if removal is staged.
- Removal requires: migration path documented, telemetry checked (no usage for 2 releases).

Style Guides

React (Frontend)

- Functional components with Hooks (avoid classes).
- TypeScript strongly encouraged (if adopted, use interfaces for props).
- Make resuable components

REST API (Backend)

- Endpoints:
  - Use nouns (e.g., **/api/users, /api/transactions**).
- HTTP Methods:
  - GET (read), POST (create), PUT/PATCH (update), DELETE.
- Responses:
  - JSON format with consistent structure:

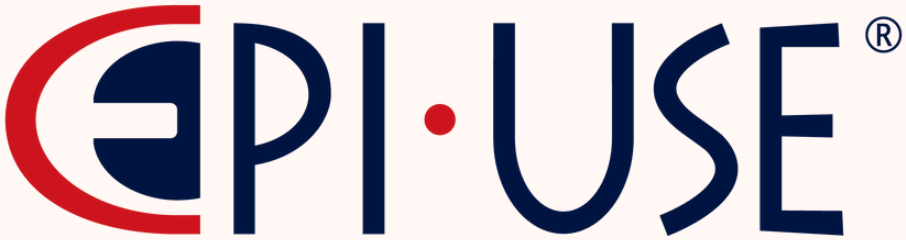
```
{
 "success": true,
 "data": { ... },
 "error": null
}
```

Error Handling:

- HTTP status codes (e.g., 400 for bad requests, 404 for not found).
- Include error messages in responses.
- Add logging for error messages and info.
- Raise errors as early as possible in your code.
- Restore state and resources after handling errors.
- Ensure try-catch blocks are used for code that is prone to runtime exceptions (**e.g., I/O operations, external API calls**).
- Keep try-catch blocks as minimal as possible to avoid catching irrelevant errors.
- Fallback mechanisms: Provide default behaviour or fallback logic if an error occurs

Gamified Finance

In Collaboration With



CodeBlooded